

Memory Allocation

2026 Semester 2 COMPSCI 340: Operating Systems

Talia Xu

Lecture 12

2.6.1

Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

Alignment Means Starting at a Multiple of N

An address is N -byte aligned when it is a multiple of N :

$$\text{address} \bmod N = 0.$$

For example, 8-byte boundaries occur every 8 bytes:

..., 0xE0, 0xE8, 0xF0, 0xF8, ...

Since $8 = 2^3$, an 8-byte-aligned address has its lowest three bits equal to zero.

Is address 0xEC 8-byte aligned?

0xEC Falls Between Two 8-Byte Boundaries

$$0xEC = 236, \quad 236 \bmod 8 = 4.$$

The remainder is not zero, so 0xEC is not 8-byte aligned.

It is four bytes after 0xE8 and four bytes before 0xF0:

$$\underbrace{0xE8}_{\text{aligned}} \longrightarrow \underbrace{0xEC}_{\text{not aligned}} \longrightarrow \underbrace{0xF0}_{\text{aligned}}.$$

Alignment may therefore leave unused bytes so the next object can begin on its required boundary.

Page Alignment Uses the Same Rule

A 4 KiB page contains $4096 = 2^{12}$ bytes. A page-aligned start address must therefore satisfy

$$\text{address} \bmod 4096 = 0.$$

Equivalently, its lowest twelve address bits are zero.

Candidate start	Lowest 12 bits	Page aligned?
0x7000	0x000	yes
0x7C00	0xC00	no
0x8000	0x000	yes

One aligned page occupies 0x7000–0x7FFF; the next page starts at 0x8000. The last three hexadecimal digits select the offset within a 4 KiB page.

Predict the Layout of This Structure

Assume char has size and alignment 1 byte, while int has size and alignment 4 bytes.

```
struct AlignmentDemo {  
    char char_a;  
    int int_a;  
    char char_b;  
    char char_c;  
    char char_d;  
    int int_b;  
    int int_c;  
};
```

Where does each field begin, and what is sizeof(struct AlignmentDemo)?

Alignment Adds Padding Between Some Fields

Field or region	Byte offset(s)	Why
char_a	0	one byte
padding	1-3	move to a multiple of 4
int_a	4-7	four-byte aligned
char_b, char_c, char_d	8-10	three bytes
padding	11	move to a multiple of 4
int_b	12-15	four-byte aligned
int_c	16-19	four-byte aligned

Therefore, under these assumptions, the structure occupies 20 bytes. The exact layout is set by the platform's ABI.

Dividing Memory into Useful-Sized Objects

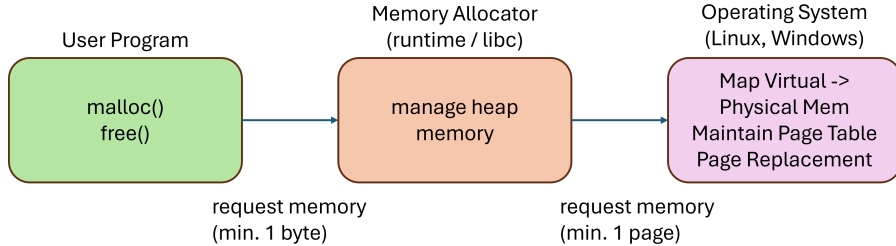
Page replacement obtains a physical frame when a page must enter RAM. Most software, however, asks for objects that are much smaller than a page.

We will move through three connected layers:

variable-size heap blocks → power-of-two page blocks → fixed-size object slots

Each layer changes the allowed block sizes so allocation can be fast while keeping wasted space manageable.

Allocation Happens at Several Layers



A user-space allocator divides virtual-memory regions for `malloc` and `free`. It asks the operating system for more virtual memory when its current heap cannot satisfy a request.

Dynamic Allocation Handles Sizes and Lifetimes Known at Runtime

A global or static object has a size and lifetime fixed when the program is built. Stack storage follows function calls.

Heap allocation is useful when:

- the required size is known only while the program runs,
- an object must outlive the function that creates it, or
- a collection must grow and shrink.

The allocator must later reuse the space returned by free.

One Heap Must Reuse Space as Requests Come and Go

Step	Operation	Heap state
1	A = malloc(24)	A:24 free space
2	B = malloc(40)	A:24 B:40 free space
3	free(A)	free:24 B:40 free space
4	C = malloc(16)	C may reuse the 24-byte hole

To do this, an allocator needs metadata, a way to find free blocks, and rules for splitting and joining blocks.

A Header Records the Size of an Allocated Block

A simple allocator places a small header immediately before the bytes returned to the program.

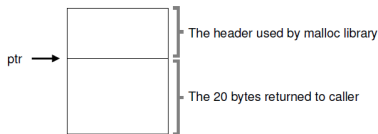


Figure 17.1: An Allocated Region Plus Header

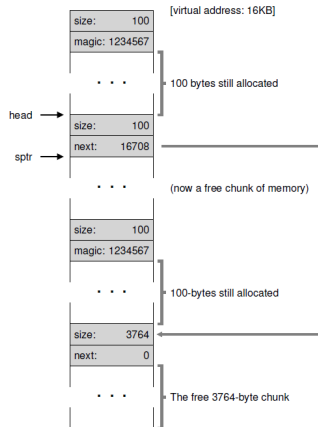
The program receives a pointer to the usable bytes. When that pointer is passed to free, the allocator can find the header and recover the full block size.

A Free List Links Blocks That Can Be Reused

A free block no longer contains program data, so the allocator can store a pointer to the next free block inside it.

The allocator keeps the list head and searches the list when satisfying a request.

An allocated block is removed from the list; a freed block is inserted again.



Splitting and Coalescing Change the Shape of Free Space

Splitting divides a large free block when a request needs only part of it.

Coalescing merges neighbouring free blocks after they are freed.

Without coalescing, adjacent holes remain separate and may be unable to satisfy a later large request.

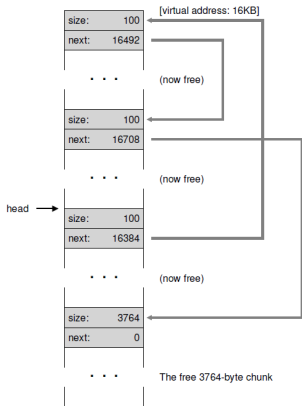


Figure 17.7: A Non-Coalesced Free List

Fragmentation Is Free Space in an Unhelpful Form

External fragmentation: free space is split across separate holes.

A 60-byte request cannot use holes of 40 and 20 bytes even though 60 bytes are free in total.

Internal fragmentation: the allocated block is larger than the request.

A 28-byte request placed in a 32-byte block wastes 4 bytes inside that block.

Variable sizes and lifetimes create external fragmentation; rounding and alignment create internal fragmentation.

Fit Policies Choose Which Free Hole to Split

First fit Use the first adequate hole and stop searching.

Best fit Use the smallest adequate hole.

Worst fit Use the largest available hole.

Best and worst fit normally search the full list. First fit is cheaper to search, but no policy can guarantee that a sequence of variable-size requests will avoid fragmentation.

First Fit Walkthrough (1/3)



First fit stops at the first hole large enough for the 40-byte request.

First Fit Walkthrough (2/3)



Continue scanning from the start for the next request.

First Fit Walkthrough (3/3)



The final 60-byte request fits exactly in the remaining hole.

Best Fit Walkthrough (1/3)

Note that blocks with a blank background and a number are free



Where do we allocate this block?

Best Fit Walkthrough (2/3)



Where do we allocate this block?

Best Fit Walkthrough (3/3)



The next block does not fit anywhere

Worst Fit Walkthrough (1/3)



Where do we allocate this block?

Worst Fit Walkthrough (2/3)



Where do we allocate this block?

Worst Fit Walkthrough (3/3)



Next block fits exactly in remaining space

The Request Sequence Changes Which Policy Looks Best

Policy	Choices for 40, 60, 60	Result
First fit	first adequate hole each time	all three fit
Best fit	60, then 100	final request fails
Worst fit	100, then a remaining 60	all three fit

Best fit leaves separate holes of 40 and 20 bytes. Together they contain 60 free bytes, but neither hole is large enough for the final request.

This is one request sequence, not a universal ranking. First fit searches less; best fit may leave tiny holes; worst fit may consume a valuable large hole.

Buddy Blocks Make Splitting and Merging Predictable

Variable-size free lists allow flexible placement, but they may require a long search and leave irregular holes.

The buddy allocator restricts every block to a power-of-two size:

32, 64, 128, 256, ...

A request is rounded up, a larger block is repeatedly split in half, and matching halves can later merge. The restriction trades some internal fragmentation for fast, structured operations.

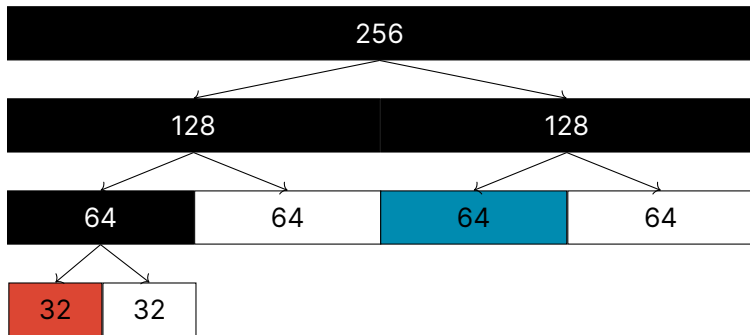
The Buddy Allocator Keeps One Free List per Block Size

For a request rounded to 2^k :

1. Check the free list for blocks of size 2^k .
2. If it is empty, take a block from the next non-empty larger list.
3. Split that block in half until one half has size 2^k .
4. Put each unused half on the free list for its size.

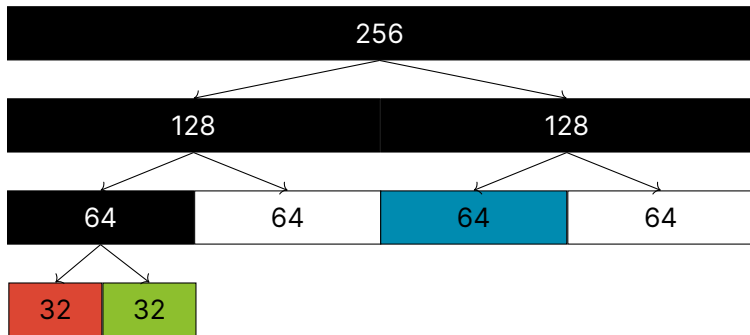
When a block is freed, its address identifies its matching buddy. If both are free, they merge and the check repeats at the next size.

Buddy Allocator Walkthrough (1/4)



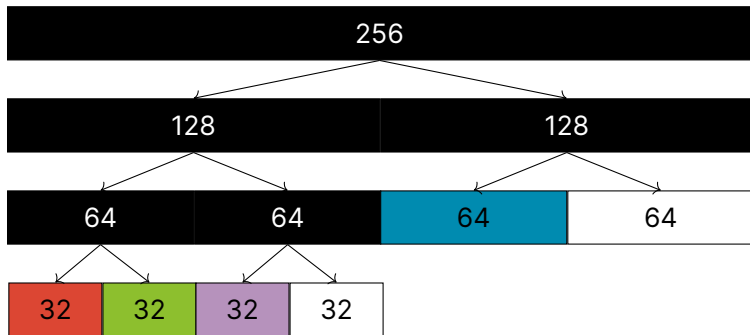
A 28-byte request rounds up to 32 bytes. Which free block can satisfy it?

Buddy Allocator Walkthrough (2/4)



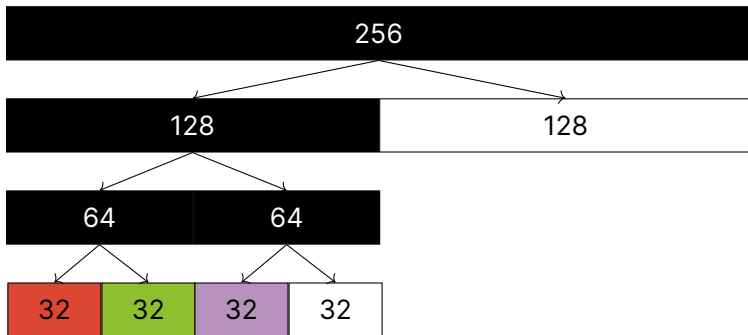
The first 32-byte block is now allocated. Where can a second 32-byte request go?

Buddy Allocator Walkthrough (3/4)



The second 64-byte block is freed. Which buddy can it merge with?

Buddy Allocator Walkthrough (4/4)



Buddy Allocation Is Fast but Rounding Wastes Space

Advantages

- one free list per size makes a suitable block quick to find,
- splitting always divides a block in half, and
- a block's address identifies the buddy used for coalescing.

Cost

- every request rounds up to a power of two, creating internal fragmentation.
- Linux uses a buddy allocator to manage free physical page blocks.

Many Kernel Objects Have the Same Size

The kernel repeatedly creates and frees many objects of the same type.

Examples include process descriptors, directory-entry objects, and network buffers. Every object of one type has the same size and alignment.

A slab allocator keeps a dedicated pool for one object type or size class. This turns repeated allocation into choosing and returning equal-sized slots.

A Slab Is a Pool of Equal-Sized Object Slots

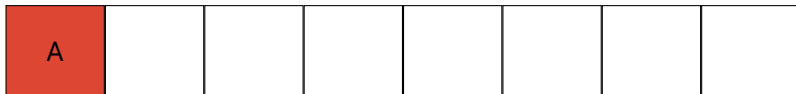
Consider a slab for 256-byte objects.

- Every slot is already 256 bytes and correctly aligned.
- A bitmap or free list records which slots are available.
- Allocation chooses one free slot.
- Freeing an object makes its slot available again.

The allocator does not search for different-sized holes inside this slab: every request uses exactly one slot.

A Slab Reuses Freed Slots

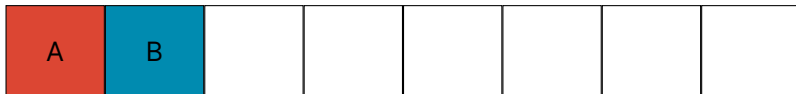
Allocate object A



Eight slots from the slab are shown.

A Slab Reuses Freed Slots

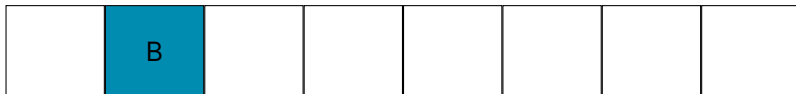
Allocate object B



The second request uses another ready-made slot.

A Slab Reuses Freed Slots

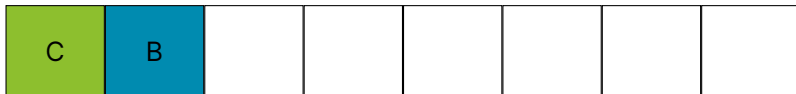
Free object A



Freeing A marks its slot available for another object.

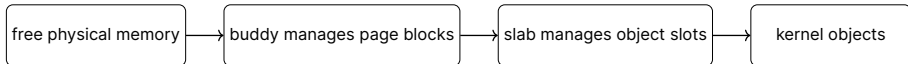
A Slab Reuses Freed Slots

Allocate object C



C immediately reuses the slot previously occupied by A.

The Kernel Builds Slab on Top of Buddy

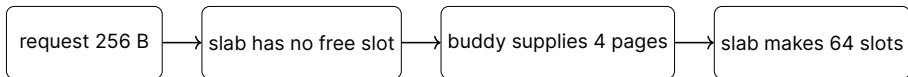


When a slab needs more backing memory, it asks buddy for one or more whole pages. Slab then divides that page block into equal-sized slots.

Buddy does not allocate the small objects. Slab does not manage free physical pages. Each allocator manages a different level.

When an Empty Slab Needs More Backing Memory

Suppose a kernel subsystem needs one 256-byte object, its slab has no free slots, and a new slab uses four backing pages.



Buddy treats the result as one order-2 block: four contiguous pages, or 16 KiB. Slab treats the same memory as sixty-four 256-byte slots.

Slab returns one slot and keeps the others ready. Buddy is not involved again until slab needs more backing pages or returns the entire block.

Buddy and Slab Track Different Things

	What it tracks	What it does in the example
Buddy	free physical page blocks	supplies one 16 KiB order-2 block
Slab	sixty-four 256-byte slots inside that block	allocates, frees, and reuses A, B, and C

When every slot in a slab becomes free, the slab allocator may eventually return the whole page block to the buddy allocator.

The relationship is therefore:

physical memory $\longrightarrow$ buddy page block $\longrightarrow$ slab slots $\longrightarrow$ kernel objects.

Each Allocation Layer Solves a Different Size Problem

Layer	Unit managed	Main technique
Heap allocator	variable-size program blocks	headers, free lists, fit
Buddy allocator	free physical page blocks	split and merge buddies
Slab allocator	small kernel objects	divide buddy blocks into slots

The layers complement one another: pages provide the raw space, buddy allocation manages page blocks, and heap or slab allocators divide larger regions into the objects software actually requests.